Machine Learning Based Detection of Objects Moving Away from FIUS Sensor with Workspace Clearance Prediction

Frankfurt University of Applied Sciences
Course: Information Technology
Module: Individual Project
Under the guidance of Prof. Dr. Andreas Pech

Akanksha Venkatesh Venkatesh Baitipuli
Matr. No.: 1568316
Email: akanksha.venkatesh-baitipuli@stud.fra-uas.de

Ayush Naresh Parmar
Matr. No.: 1565427
Email: ayush.parmar@stud.fra-uas.de

***Abstract*—Ultrasonic sensors are commonly used in industrial automation for short range object detection due to their low cost and robustness to different lighting conditions. This paper presents AWY-1, a machine learning system built on real distance measurements from the Frankfurt Intelligent Ultrasonic Sensor (FIUS) that goes beyond simple presence detection. The primary focus is detecting objects moving away from the sensor and predicting Time-to-Clear (TtC), which gives the time in seconds until the object exits the defined safety zone. Approaching and Stationary motion are also classified to provide complete motion state context. Data were collected across two recording sessions covering three object types: a human, a chair, and a metal plate under daylight and night conditions at slow, medium, and fast speeds. Five machine learning classifiers were trained and compared. Random Forest achieved the best classification accuracy at 85.93%, meeting the project target of $\geq$85%, and produced the lowest TtC prediction error among all models. An interactive web application demonstrates the trained models in a simulated warehouse safety corridor, showing live zone monitoring, motion classification, and TtC driven decisions across all three motion scenarios.**

***Index Terms*—machine learning, ultrasonic sensing, motion classification, Random Forest, Time-to-Clear prediction, workspace safety, FIUS sensor, object detection**

I. Introduction

Industrial automation increasingly relies on shared workspaces where robots and human operators work in close proximity. Ensuring safety in these environments requires continuous monitoring of the space around robotic systems detecting not just whether a person is present, but understanding their behaviour and anticipating when the shared space will be safe to resume operation [1]. Ultrasonic distance sensors are a natural choice for this monitoring role: they are low-cost, robust to varying lighting conditions, and already widely deployed in industrial proximity detection systems [2].

Current ultrasonic based safety systems, however, operate as simple threshold triggers. When the measured distance falls below a fixed value, the robot stops. When it rises above that value, the robot resumes. This binary approach cannot answer two questions that matter most for efficient workspace management: is the object stationary or already leaving, and if it is leaving, when will the zone be clear? A person standing still at 0.85 m and a person walking away at 0.85 m produce identical instantaneous sensor readings. The robot treats both the same and waits with no information about whether clearance is two seconds away or two minutes away. In busy manufacturing environments, this unnecessary waiting time directly reduces throughput without any corresponding safety benefit.

This limitation motivates AWY-1. Rather than treating each distance reading in isolation, AWY-1 analyses temporal windows of consecutive sensor readings and extracts motion features that distinguish direction, speed, and consistency of movement. The system addresses two tasks simultaneously: classifying object motion into Approaching, Away, or Stationary, and predicting Time-to-Clear (TtC) for Away objects still inside the 0.90 m safety zone. The primary focus is the Away class and TtC prediction detecting that an object is leaving and estimating how soon the zone will be free. Approaching and Stationary are classified to provide complete motion state context and to enable meaningful comparison with rule-based baselines. While prior work has approached similar problems using camera-ultrasonic sensor fusion [3] or multi-sensor arrays [2], AWY-1 operates from a single scalar distance stream the output of one horizontally mounted Frankfurt Intelligent Ultrasonic Sensor (FIUS).

Although the original project proposal specified a simulation-based approach, real distance measurements were collected from the FIUS sensor across two laboratory sessions covering three object types (human, chair, metal plate), two lighting conditions, and three motion speeds. Real sensor data introduces genuine measurement noise, multipath reflections, and object-specific reflection characteristics that simulation cannot fully replicate. The original 3.0 m zone threshold was also revised to 0.90 m because laboratory space limited

recordings to approximately 3.0 m on Day 2 and only 1.08 m on Day 1 [4]. Five classifiers were evaluated against rule-based baselines. Random Forest achieved the highest accuracy at 85.93%, meeting the $\geq$85% project target, and produced the lowest TtC prediction error. An interactive Streamlit application demonstrates the trained system in a simulated warehouse safety corridor across all three motion scenarios.

A. Objectives

1) Detect objects moving away from the sensor and classify all motion states with accuracy $\geq$85%.
2) Predict Time to Clear with MAE < 0.6 s for Away objects inside the safety zone.
3) Demonstrate that machine learning outperforms rule based baselines on both tasks.
4) Evaluate generalisation across object types, speeds, and lighting conditions using real FIUS sensor recordings.

II. Related Work

The motivation for AWY-1 sits at the intersection of two research areas: safe human robot collaboration and ultrasonic sensing. Early work on human robot collaborative manipulation showed that anticipating human motion rather than reacting to it after the fact substantially improves both safety and workflow efficiency in shared workspaces [1]. More recently, digital twin based evaluation frameworks for proactive workspace clearance and collision avoidance have demonstrated that predicting when a human will exit an operational zone is a tractable and practically valuable problem [5]. AWY-1 approaches the same problem from the sensor side, using a single ultrasonic distance stream rather than camera or joint space observations.

Ultrasonic sensors are widely used for short range object detection in robotics and automation because they are low cost and unaffected by ambient lighting. Raw readings from these sensors, however, carry more error than manufacturers typically claim. Hunter [4] tested an HC-SR04 sensor against tape measure ground truth at 30 distances and found a mean absolute error of 4.97 mm, compared to the claimed maximum of approximately 3 mm gap confirmed statistically by bootstrap and confidence interval tests. Error also grew at longer measurement distances. The FIUS sensor used in AWY-1 differs in design, but the underlying issue is the same: raw distance readings from low cost ultrasonic hardware require correction before use. This motivated the hard clipping at $[0.05, 5.0]$ m and the rolling Median Absolute Deviation filter applied to all FIUS recordings before feature extraction.

The need for preprocessing in range based sensing is also documented outside the ultrasonic domain. Wang et al. [6] modelled distance measurements from ranging sensors as a nonstationary stochastic process and developed online methods for noise attenuation, outlier removal, and dropout correction. Their central finding is that raw range readings contain enough uncertainty that downstream analysis on uncorrected data is unreliable. Although their system targets positioning rather than motion classification, the same problem applies to the FIUS distance stream in AWY-1, where spike outliers from multipath reflections corrupt window level features if not removed first.

Machine learning has been applied to improve how ultrasonic signals are interpreted under noise. Mohamed et al. [7] showed that a trained model detects ultrasonic echoes more reliably than fixed threshold methods in noisy environments. In a related study [8], a CNN was applied to suppress noise in automotive ultrasonic sensor waveforms, benchmarked against a Discrete Wavelet Transform on both simulated and real industrial sensor data. The CNN achieved comparable noise suppression while introducing significantly less distortion to the underlying signal, and continued recovering the signal at low SNR where the wavelet method failed entirely. Both studies operate at the raw waveform level a different abstraction than the processed scalar distance values the FIUS produces. The broader finding still holds: learned models adapt to variable acoustic conditions better than fixed rules, which supports choosing Random Forest over velocity threshold baselines for motion classification on FIUS data.

Nesti et al. [2] built an object detection pipeline for autonomous vehicles using a multi sensor ultrasonic array, converting echo data into voxelized 3D point clouds and bird's eye view images for deep learning inference. Their results confirmed that ultrasonic sensors perform best in near range, low speed scenarios, with highest detection accuracy for objects within 3 m. AWY-1 uses a single FIUS sensor producing scalar distance readings rather than a multi sensor echo array, so the architectures differ substantially. The near range performance characteristic they report is consistent with the AWY-1 setup, where the zone boundary sits at 0.90 m.

The closest prior work to AWY-1 in task formulation is Tapu et al. [3], who built a wearable assistive device for visually impaired people combining ultrasonic sensors and a smartphone camera to classify obstacles as approaching or moving away. Their system achieves an F-score above 82% for obstacle classification. However, it requires two sensing modalities the camera provides motion vectors that the ultrasonic sensor alone cannot supply. It also does not classify stationary objects as a separate state, and provides no estimate of when a moving object will exit a defined zone. AWY-1 addresses all three gaps: it classifies Approaching, Away, and Stationary from a single FIUS distance stream alone, and adds Time-to-Clear regression using real recordings across three object types, three motion speeds, and two lighting conditions.

III. System Overview

AWY-1 takes raw CSV distance recordings from a single horizontally mounted FIUS sensor and produces two outputs: a motion class label per 300 reading window and, for Away windows where the object is still within the 0.90 m safety zone, an estimate of Time-to-Clear in seconds. The full pipeline runs in six sequential stages, illustrated in Fig. 1.

Fig. 1. AWY-1 system pipeline from raw CSV input to trained model output.

A. Project Workflow

Each stage in Fig. 1 is described below.

1) *Data ingestion:* Each CSV file is loaded by reading column index 10, which stores the distance in metres. One file corresponds to one recorded trajectory with a single class label.
2) *Signal cleaning:* Readings outside $[0.05, 5.0]$ m are clipped first 0.05 m is below the sensor's minimum reliable range and 5.0 m exceeds its practical maximum. A rolling Median Absolute Deviation filter then removes spike outliers. A reading at index i is removed if:

$$\frac{|x_i - \tilde{x}_i|}{\text{MAD}_i} \geq 6 \tag{1}$$

where $\tilde{x}_i$ is the rolling median and MAD_i is the rolling MAD over a 21 reading window centred on i. MAD is preferred over standard deviation because it is not inflated by the outliers it removes [4].The threshold of 6 was chosen because values below 4 removes genuine slow-motion readings while values above 8 retained visible spike artifacts in the cleaned signal.
3) *Feature extraction:* A window of 300 consecutive cleaned readings (≈7.9 s at 38 Hz) slides over each signal with a stride of 50 readings for training and 100 for testing. Each window produces 16 features described in Section IV-C.
4) *Ground truth labelling:* Classification labels come from each file's motion category. TtC ground truth is computed per file for Away class recordings only: for each reading i where $d_i < 0.90$ m, TtC is the elapsed time until the first reading $j > i$ where $d_j \geq 0.90$ m. Computing this per file prevents TtC values from spanning two separate recordings.
5) *Training and evaluation:* The first 75% of readings in each file form the training partition; the last 25% form the test partition. This temporal ordering ensures no test window overlaps with training data from the same trajectory, which would otherwise inflate reported accuracy. Models are trained and evaluated using accuracy, F1-score, MAE, and RMSE.
6) *Output and deployment:* Trained models are saved as `.joblib` files. A Streamlit web application loads the saved models and runs live inference, displaying the predicted motion class and TtC estimate at each window step. This application is described in detail in Section VII-D.

B. System Architecture

Fig. 2 shows the complete AWY-1 system architecture. The system is divided into three layers: data layer, processing layer, and output layer.

In Data layer the FIUS sensor mounted horizontally in the laboratory produces one distance reading every 26.3 ms at 38 Hz. Each recording session produces one CSV file per trajectory. The data layer loads each file and selects column index 10, which stores the distance in metres. The Processing layer consists of five sequential stages. The signal cleaner applies hard clipping at $[0.05, 5.0]$ m followed by a rolling MAD filter to remove spike outliers. The feature extractor slides a 300-reading window over the cleaned signal and computes 16 kinematic, statistical, and directional features per window. The labelling stage assigns motion class labels from the file metadata and computes TtC ground truth per file for Away-class recordings. The train/test split divides each file temporally 75% for training and 25% for testing. Finally, `StandardScaler` is fitted on the training partition only and applied to Logistic Regression, KNN, SVM, and Linear Regression, while Decision Tree and Random Forest receive raw unscaled features.

The output layerhas two trained models form the output layer. The Random Forest classifier receives the 16-feature vector and outputs one of three motion class labels: Approaching, Away, or Stationary. If the prediction is Away and the object is inside the 0.90 m zone, the Random Forest regressor receives the same feature vector and outputs a TtC estimate in seconds. Both models are saved as `.joblib` files and loaded by the Streamlit demo application for live inference without retraining. The demo displays the predicted motion state, TtC value, and safety action on a live dashboard.

C. Application Scenario

The primary scenario is a fixed, horizontally mounted FIUS sensor monitoring a defined safety zone in front of it. The zone is occupied when a measured distance falls below 0.90 m and clear when it is at or above that value.

A threshold only sensor can report whether the zone is currently occupied but cannot determine what the object is doing. A person standing still at 0.85 m and a person walking away at 0.85 m produce the same instantaneous reading. AWY-1 makes the distinction: the Away classifier detects that the object is leaving, and the TtC regressor estimates how many seconds remain until the zone clears.

This information is useful in any application where a system needs to know not just that a zone is occupied but whether it will be free soon. Practical contexts include collaborative robot workspaces, automated conveyor systems, and corridor access monitoring, where knowing the motion state and predicted clearance time allows downstream systems to act proactively rather than waiting for the threshold to flip.

The interactive demo application illustrates one such scenario a simulated warehouse safety corridor where the system monitors zone state, classifies motion, and outputs a clearance time estimate in real time.

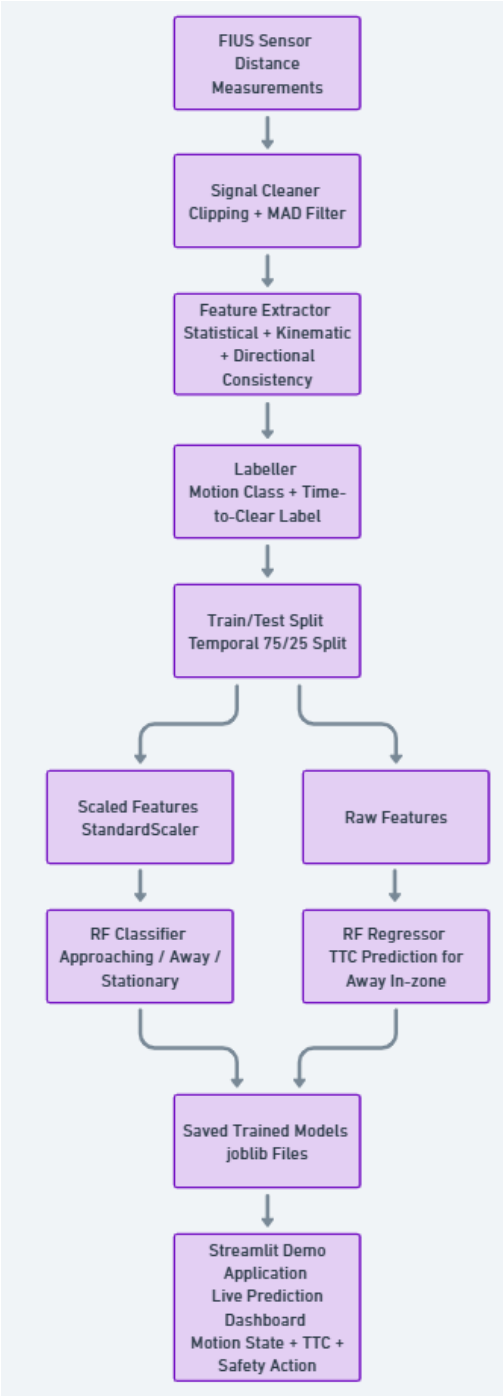

Fig. 2. AWY-1 system architecture from raw FIUS sensor input through signal cleaning, feature extraction, labelling, and model training to the live prediction dashboard.

IV. Dataset and Experimental Setup

A. Data Source

Data was collected over two recording sessions using the Frankfurt Intelligent Ultrasonic Sensor (FIUS), mounted horizontally and producing one distance measurement every 26.3 ms at a rate of 38 Hz. Each recording is stored as a CSV file; column index 10 contains the distance in metres. Although the original project proposal specified a simulation based approach using synthetic distance trajectories, real distance measurements were collected from the FIUS sensor across two laboratory sessions. Real sensor data introduces genuine measurement noise, multipath reflections, and object specific reflection characteristics that simulation cannot fully replicate, providing a more realistic training and evaluation dataset than synthetic data would allow.

Day 1 consists of single condition trajectories recorded using a single object type (a chair) under controlled indoor conditions with fixed lighting. Day 2 introduces variation across three object types (human, chair, metal plate), two lighting conditions (daylight and night), and three motion speeds (slow, medium, fast). The speed labels slow, medium, and fast correspond to approximate velocity ranges estimated from the linear slope of each trajectory. Slow motion corresponds to approximately 0.01–0.05 m/s, medium to 0.05–0.15 m/s, and fast to 0.15–0.50 m/s. These ranges were estimated from the least squares velocity feature computed across each file and are consistent across both object types and lighting conditions in Day 2.

The original project proposal specified a zone threshold of 3.0 m based on a simulated scenario. Real data collection was constrained by the available laboratory space, which limited recordings to approximately 3.0 m on Day 2 and only 1.08 m on Day 1. Since the trained models have not observed Away motion beyond these distances, applying a 3.0 m threshold would place the zone boundary at the edge of the training data distribution, making both classification and TtC prediction unreliable near that boundary. The threshold was therefore set to 0.90 m, a value that all Away trajectories in the dataset clearly cross, ensuring reliable TtC ground truth across every recording session.

Before training, each file was inspected for label consistency. Recordings where the measured motion was inconsistent with the assigned class label were excluded for example, Away files where the object was measured moving toward the sensor rather than away. Stationary files with excessive distance variance were also removed. Table I shows the dataset after these exclusions.

TABLE I
Dataset Composition

Class	**Total**	**Session**	
		Day 1	*Day 2*
Approaching	15,988	9,055	6,933
Away	37,374	9,125	28,249
Stationary	21,550	12,397	9,153
Total	**74,912**	**30,577**	**44,335**

The Away class is overrepresented because Away trajectories in the FIUS recordings are longer in duration than Approaching trajectories, producing more windows per file. This deviates from the balanced dataset proposed but reflects the natural structure of the real recordings.

B. Classes

Three motion classes are defined based on how the measured distance changes over time. Fig. 3 shows representative distance profiles and distributions for each class from the FIUS recordings.

The Approaching class shows a steadily decreasing distance profile the object moves toward the sensor and into the zone. The Away class starts inside the zone and moves outward, crossing the 0.90 m boundary. The Stationary class remains at a fixed position with a near flat profile and a narrow distance distribution. The overlap between Approaching and Away distributions in the 0.5 - 1.5 m range is visible in the bottom row both classes occupy the same distance range at different points in their trajectories, which is precisely why instantaneous distance alone cannot classify motion direction and a temporal window approach is needed.

1) Approaching (Class 0): the object moves toward the sensor; distance decreases over the window.
2) Away (Class 1): the object moves away from the sensor; the distance increases over the window.
3) Stationary (Class 2): the object remains at a fixed position; distance is approximately constant.

A zone threshold of $d_{thresh} = 0.90$ m defines the boundary between occupied and clear states. An Away object still within the zone ($d < 0.90$ m) also receives a Time-to-Clear ground truth value, computed as described in Section III-A.

C. Input Features

Each 300 reading window produces 16 features organised into three groups.

Statistical features describe the distance distribution within the window: mean, standard deviation, minimum, maximum, and range. These separate stationary objects which have near zero standard deviation and range from moving ones.

Kinematic features capture motion direction and speed: linear velocity from a least squares fit over the window, acceleration from the velocity difference between the first and second half windows, net displacement ($w_{N-1} - w_0$), normalised velocity, and trend noise from the linear fit residuals.

Directional consistency features count the fraction of consecutive reading pairs where distance increases (`frac_increasing`) and where it decreases (`frac_decreasing`). These features are robust to absolute distance variation and their importance in the trained classifier is discussed in Section VII-A.

Three positional features record distance at the window start, centre, and end. A binary `in_zone` flag marks windows where the mean distance falls below d_{thresh}.

D. Data Preprocessing

The raw CSV files contain no missing values. Spike outliers are removed by the rolling MAD filter in Stage 2; flagged readings are dropped before windowing rather than imputed, so no gaps are introduced into the distance stream.

Features are standardised to zero mean and unit variance using `StandardScaler` fitted on the training partition only, then applied to the test partition. Logistic Regression and Linear Regression use standardised features. Random Forest receives raw features since decision trees are invariant to monotonic transformations.

Class labels are stored as integers (0, 1, 2). No one hot encoding is needed since all models accept integer labels directly.

The dataset is split temporally per file: the first 75% of readings form the training partition and the last 25% the test partition. Windows are extracted from each partition independently so no window crosses the split boundary. A random split was deliberately avoided on time ordered sensor data, random folds allow windows from the same trajectory to appear in both sets, leaking temporal information and inflating reported accuracy. Table II shows the resulting window counts.

TABLE II
WINDOW COUNTS AFTER TEMPORAL SPLIT

Task	Train	Test
Classification (all 3 classes)	1,122	135
Regression (Away in zone only)	169	57

V. METHODOLOGY

A. Model Selection

The original project proposal specified Logistic Regression and Random Forest as the two classifiers. Three additional models Decision Tree, K-Nearest Neighbors, and Support Vector Machine were included to provide a more comprehensive comparison across different decision boundary types. This allows the results to show not just that Random Forest outperforms the proposed baseline, but where it sits relative to a range of established classifiers. Two rule based baselines are also evaluated as a performance floor. For TtC regression, Linear Regression and Random Forest are trained as proposed.

Logistic Regression fits a linear decision boundary in the 16 dimensional feature space with L2 regularisation ($C = 1.0$, LBFGS solver). It serves as the linear ML baseline if the three classes were linearly separable, Logistic Regression would be sufficient and Random Forest unnecessary.

Decision Tree builds a single axis aligned tree with no depth constraint. Since Random Forest is an ensemble of decision trees, comparing the two directly shows how much bootstrap aggregation and random feature selection help on this data.

K-Nearest Neighbors ($k = 5$) classifies each test window by majority vote among its five nearest neighbours in standardised feature space. It makes no assumption about the shape of the decision boundary.

Support Vector Machine uses an RBF kernel ($C = 1.0$) to find a maximum margin boundary in a higher dimensional projection of the feature space.

Random Forest trains an ensemble of 500 decision trees via bootstrap aggregation with random feature selection at each split (`min_samples_split = 3`). Raw unscaled features are used since tree based methods are invariant to monotonic transformations.

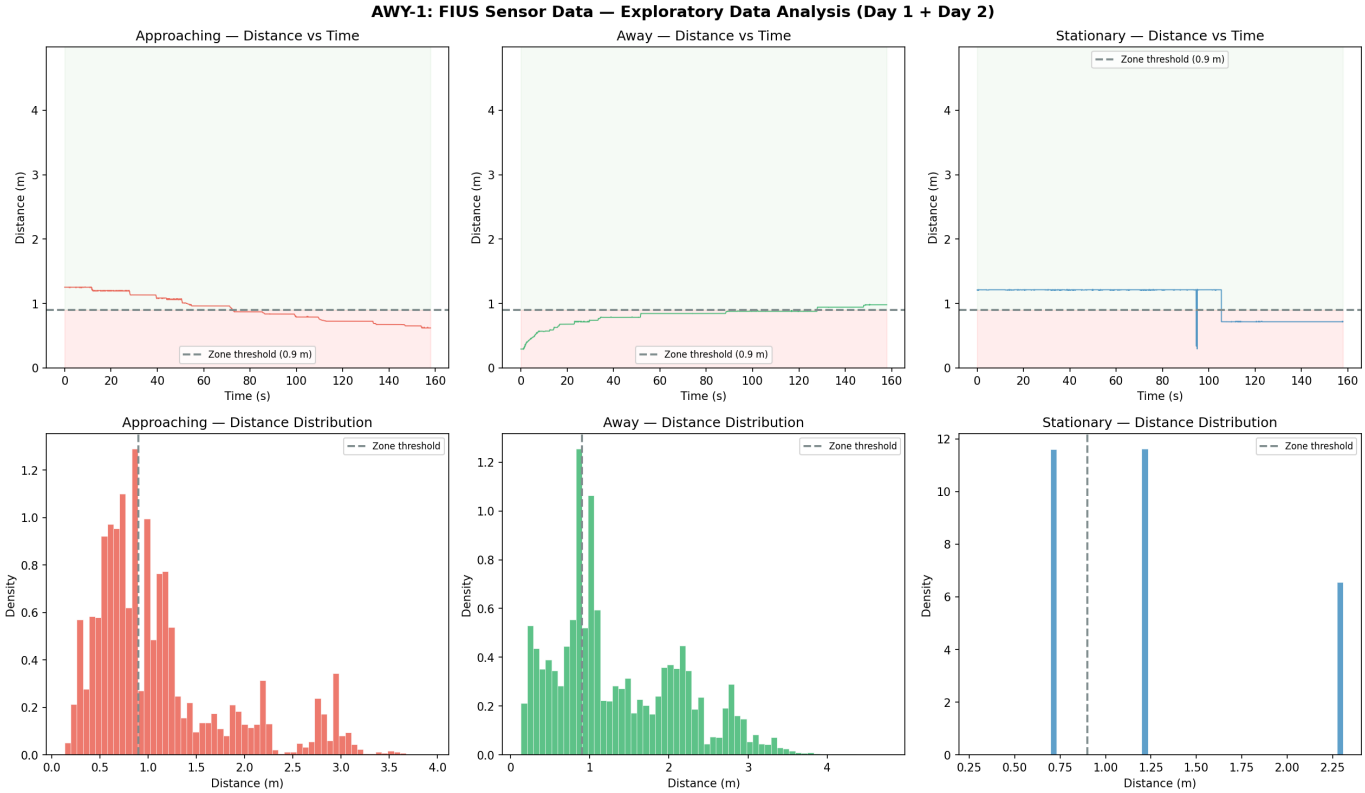

Fig. 3. FIUS sensor recordings for all three motion classes. Top row: distance vs time showing how each class behaves relative to the 0.90 m zone boundary (dashed line). Bottom row: corresponding distance distributions.

Baseline 1 (threshold only) always predicts Stationary regardless of input and cannot distinguish motion direction. Baseline 2 (trend based) predicts Away if velocity $> +0.02$ m/s, Approaching if velocity < -0.02 m/s, and Stationary otherwise.

For regression, Linear Regression with ordinary least squares is the baseline and Random Forest (500 trees, `min_samples_split = 3`) is the primary model. Predictions below zero are clipped to zero since negative TtC has no physical meaning.

B. Training Procedure

The classification training set contains 1,122 windows; the test set contains 135. The regression training set contains 169 Away in zone windows; the test set contains 57.

For classification, all splits are temporal per file: the first 75% of each file's readings form the training partition and the last 25% the test partition. No window crosses the split boundary. A random split was deliberately avoided since on time ordered sensor data, random folds allow windows from the same trajectory to appear in both sets, inflating accuracy by leaking temporal information.

For regression, the 226 in zone Away windows from all files are pooled and randomly shuffled before splitting 75/25. A strict per file temporal split was not used here because the total regression window count is small enough that a temporal split would produce an unrepresentative test set. This introduces temporal leakage risk, as windows from the same recording may appear in both training and test sets the exact pattern carefully prevented for the classification task. This is a known constraint of the limited regression dataset size and is acknowledged as a limitation in Section VIII.

Logistic Regression, KNN, SVM, and Linear Regression use features standardised by `StandardScaler` fitted on the training partition only. Decision Tree and Random Forest use raw unscaled features. No hyperparameter search was performed. The values stated above `n_estimators = 500`, `min_samples_split = 3`, `n_neighbors = 5`, `C = 1.0` were set by design based on the problem size, not by cross validated tuning. The class imbalance between Away (558 training windows) and Approaching (238 windows) was not corrected by weighting or oversampling; its effect on per class recall is discussed in Section VII.

C. Evaluation Metrics

For motion classification, the primary metric is accuracy against the $\geq$85% project target:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{2}$$

Accuracy alone can be misleading when classes are imbalanced, so per class precision, recall, and F1-score are also reported to check whether the model performs consistently across all three classes.

For TtC regression, Mean Absolute Error is the primary metric because it expresses prediction error directly in seconds:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{3}$$

RMSE is reported alongside MAE. Because RMSE squares errors before averaging, large individual mispredictions contribute disproportionately. Comparing the two reveals whether errors are distributed evenly or driven by a small number of difficult windows.

D. Mathematical Formulation

The predicted class is:

$$\hat{y} = \arg\max_{c\in\mathcal{C}} P(c \mid \mathbf{x}), \quad \mathcal{C} = \{0, 1, 2\} \tag{4}$$

where $\mathbf{x}$ is the 16 dimensional feature vector from one 300 reading window.

Linear velocity is the least squares slope over the window scaled by the measurement rate. It is the most direct indicator of motion direction, but for slow moving objects the slope is small relative to sensor noise, which is why the directional consistency features below were added to reinforce the signal:

$$v = \hat{\beta}_1 \times f_s, \quad f_s = 38\,\text{Hz} \tag{5}$$

Directional consistency counts the fraction of consecutive step pairs that increase or decrease. These two features proved to be the most important in the trained Random Forest, accounting for 24.7% of total feature importance, and are more robust than velocity for slow motion because they do not depend on displacement magnitude:

$$f_{inc} = \frac{1}{N-1}\sum_{i=0}^{N-2}\mathbf{1}[w_{i+1} > w_i] \tag{6}$$

$$f_{dec} = \frac{1}{N-1}\sum_{i=0}^{N-2}\mathbf{1}[w_{i+1} < w_i] \tag{7}$$

TtC ground truth at reading index i is defined as the time until the object first exits the zone:

$$\text{TtC}(i) = \frac{j^* - i}{f_s} \tag{8}$$

$$j^* = \min\{j > i : d_j \geq d_{thresh}\} \tag{9}$$

This is computed per file so that TtC values never span two separate recordings. If no crossing exists within the file, no TtC label is assigned and the window is excluded from the regression dataset.

VI. Implementation

A. Software Tools

The pipeline is implemented as a standalone Python 3 script (`awy1_project.py`), executable from any standard Python 3.8+ environment. All models, preprocessing steps, and visualisations use standard open source libraries with no custom compiled dependencies. Table III lists the libraries used.

TABLE III
Libraries Used in the AWY-1 Pipeline

Library	Purpose
NumPy	Array operations, feature computation
Pandas	CSV loading, DataFrame management
Scikit-learn	All models, scaling, metrics
Matplotlib	Result plots and figures
Seaborn	Confusion matrix heatmaps
Joblib	Model serialisation to disk
Plotly	Corridor visualisation in demo app
Streamlit	Interactive demo application

Trained models are saved as `.joblib` files immediately after fitting and loaded by the Streamlit demo without retraining. All output files figures, tables, and models are written to a `results/` directory on each pipeline run, so the entire output set is regenerated from a single script execution.

B. Code Structure

The script begins by defining all global constants including the zone threshold (0.90 m), window size (300 readings), stride values for training and testing, and the measurement rate (38 Hz). Fig. 4 shows these configuration values as defined in the script. The `TRAIN_FRAC = 0.75` constant controls the temporal train/test split the first 75% of each file's readings form the training partition and the last 25% form the test partition. This value is applied consistently across all files and both tasks so that no test window ever overlaps with training data from the same trajectory.

```
ZONE_THRESHOLD    = 0.90   # metres
WINDOW_SIZE       = 300    # readings per feature window (~7.9 s)
STRIDE_TRAIN      = 50
STRIDE_TEST       = 100
STRIDE_REG        = 50
TRAIN_FRAC        = 0.75

MEASUREMENT_RATE  = 38     # Hz
TIME_PER_READING  = 1.0 / MEASUREMENT_RATE
```

Fig. 4. Global configuration constants from the AWY-1 script including zone threshold, window size, stride values, measurement rate, and the 75/25 train/test split fraction.

Feature extraction is handled by `extract_features()`, which takes a single 300 reading window and returns all 16 features as a dictionary. The 16 feature names are defined in `FEATURE_COLS` as shown in Fig. 5. A second function `build_windows()` slides this extraction across each file at the specified stride to produce the full windowed dataset for training and testing.

```python
FEATURE_COLS = [
    'mean_dist', 'std_dist', 'min_dist', 'max_dist', 'range_dist',
    'velocity_ms', 'acceleration_ms2', 'trend_noise',
    'start_dist', 'end_dist', 'center_dist',
    'net_displacement', 'norm_velocity',
    'frac_increasing', 'frac_decreasing',
    'in_zone',
]


def extract_features(distances, start_idx, win=WINDOW_SIZE):
    w = distances[start_idx: start_idx + win]
    t = np.arange(win, dtype=float)
```

Fig. 5. Feature column definitions and the `extract_features()` function header from the AWY-1 script.

TtC ground truth is computed by forward scanning each Away file independently. For every reading at index i where $d_i < d_{thresh}$, the code scans forward to find the first index j where $d_j \geq d_{thresh}$ and stores $(j - i) \times t_{per_reading}$ as the TtC value in a lookup dictionary keyed by global reading index. Fig. 6 shows this implementation. Computing TtC per file rather than on the concatenated Away array ensures that TtC values never span two separate recordings.

Once features and labels are ready, `StandardScaler` is fitted on the training partition only and applied to the test partition. Logistic Regression, KNN, SVM, and Linear Regression receive standardised features. Decision Tree and Random Forest receive raw features since tree based methods are invariant to monotonic transformations. All five classifiers are then trained in sequence followed by the two regressors. The Random Forest classifier and regressor are saved to `results/models/` immediately after fitting. As Random Forest achieved the highest classification accuracy and lowest TtC prediction error among all models evaluated, its saved `.joblib` files are the models loaded by the Streamlit demo application for live inference.

```python
ttc_lookup = {}
offset_away = 0

for arr in raw_data_files[LABEL_AWAY]:
    n = len(arr)
    for i in range(n):
        if arr[i] >= ZONE_THRESHOLD:
            continue
        for j in range(i + 1, n):
            if arr[j] >= ZONE_THRESHOLD:
                ttc_lookup[offset_away + i] = (j - i) * TIME_PER_READING
                break
    offset_away += n

away_dist = raw_data[LABEL_AWAY]
ttc_idx   = np.array(sorted(ttc_lookup.keys()))
ttc_val   = np.array([ttc_lookup[i] for i in ttc_idx])

print(f"\nTtC ground truth computed:")
print(f"  Readings with TtC : {len(ttc_idx)} / {len(away_dist)}")
if len(ttc_val) > 0:
    print(f"  TtC range         : [{ttc_val.min():.2f}, {ttc_val.max():.2f}] s")
    print(f"  TtC mean ± std    : {ttc_val.mean():.2f} ± {ttc_val.std():.2f} s")
```

Fig. 6. TtC ground truth computation from the AWY-1 script. For each in zone reading at index i, the code finds the first j where $d_j \geq d_{thresh}$ and stores the elapsed time as the TtC value.

VII. Results and Discussion

A. Model Comparison and Best Performing Model

Table IV summarises classification accuracy across all five models and two baselines. The results follow a clear pattern: rule based methods plateau below 60%, linear ML models reach the mid 60s, tree based and distance based methods cluster around 70 -78%, and only the Random Forest ensemble crosses the 85% project target.

The threshold only baseline scores 32.59% because it always predicts Stationary it gets every Stationary window correct but misses all Approaching and Away windows entirely. The trend based baseline improves to 57.78% by using velocity sign, but its fixed ± 0.02 m/s deadband fails on slow trajectories where velocity is near zero and classification becomes ambiguous. Logistic Regression reaches 67.41%, confirming that the three motion classes are not linearly separable. KNN (71.11%) and SVM (66.67%) both underperform expectations.

TABLE IV
Classification Results All Models

Model	Accuracy	Precision	Recall	F1
Threshold only (B1)	32.59%	10.62%	32.59%	16.02%
Trend based (B2)	57.78%	77.74%	57.78%	56.06%
Logistic Regression	67.41%	68.73%	67.41%	66.85%
Decision Tree	78.52%	81.32%	78.52%	79.45%
K-Nearest Neighbors	71.11%	74.12%	71.11%	70.85%
SVM (RBF kernel)	66.67%	70.56%	66.67%	66.20%
Random Forest	**85.93%**	**86.85%**	**85.93%**	**86.25%**

The Decision Tree at 78.52% is the most informative intermediate result a single tree overfits on the small training set of 1,122 windows. Random Forest resolves this by aggregating 500 trees, gaining 7.4 percentage points to reach 85.93% and meeting the project target. Fig. 7 shows this progression across all models. Table V shows TtC regression results. The trend based baseline produces an MAE of 313.04 s because its formula $\text{TtC} = (d_{thresh} - d)/v$ collapses when velocity is near zero. Linear Regression reduces this to 16.22 s but still fails the 0.6 s target because TtC is a non linear function that a linear model cannot capture. Random Forest achieves the lowest MAE at 6.38 s. Fig. 8 shows the MAE comparison across all three regression models the scale difference between the trend based baseline and the ML models confirms that rule based TtC estimation is fundamentally unreliable for slow moving objects.

Random Forest is the best performing model on both tasks. Its feature importance scores, shown in Fig. 9, reveal why. The top two features are `frac_increasing` (0.1418) and `std_dist` (0.1217), followed by `net_displacement` (0.1058) and `frac_decreasing` (0.1050). The two directional consistency features together account for 24.7% of total importance the strongest contribution of any feature pair. This confirms that motion direction is the primary discriminator, not absolute distance. The `in_zone` flag (0.003) and `acceleration_ms2` (0.006) contribute negligibly and could be removed without affecting performance.

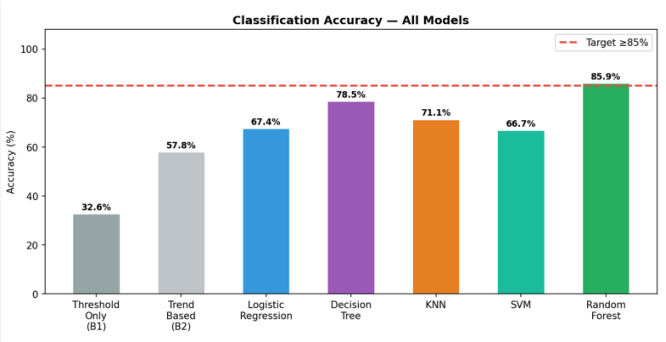

Fig. 7. Classification accuracy across all models. Only Random Forest crosses the ≥85% project target (dashed red line) at 85.9%.

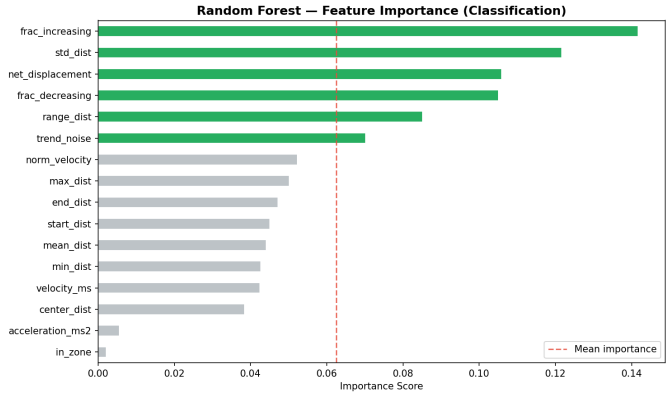

Fig. 9. Random Forest feature importance scores for the classification task. Green bars exceed the mean importance threshold (dashed red line).

TABLE V
TTC REGRESSION RESULTS

Model	MAE (s)	RMSE (s)
Trend based (B2)	313.04	1302.59
Linear Regression	16.22	24.61
Random Forest	**6.38**	**10.82**

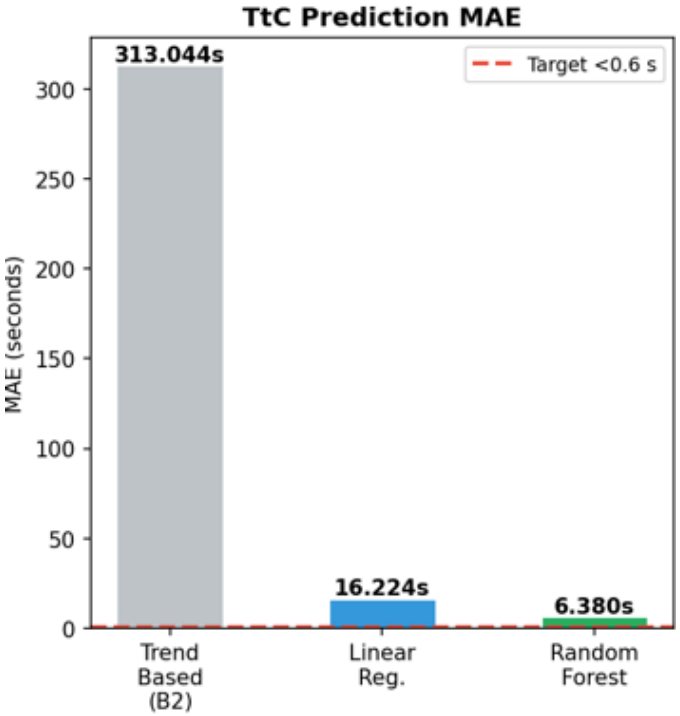

Fig. 8. TtC MAE comparison across regression models. The trend based baseline (313.04 s) collapses for slow objects. Random Forest achieves the lowest MAE at 6.38 s. Dashed red line shows the 0.6 s target.

Fig. 10. Random Forest per class Precision, Recall, and F1-score. Stationary achieves 100% precision. Away is the most balanced class. Approaching has the lowest precision at 69.2% due to confusion with slow Away trajectories.

Fig. 10 shows the per class precision, recall, and F1-score for Random Forest. Stationary achieves the strongest performance with 100% precision and 90.9% recall the model never wrongly predicts a moving window as Stationary. This is the safety critical direction of error for the zone monitoring use case: the system will not falsely declare the workspace clear when an object is standing still. Away achieves balanced precision and recall at approximately 86%, making it the most reliably detected motion class. Approaching has the lowest precision at 69.2%, meaning some Away windows are incorrectly labelled as Approaching typically slow Away trajectories where the directional signal is weak within the 7.9 s window.

Fig. 11 shows the normalised confusion matrix for Random Forest. Away recall is 85.7% the remaining 14.3% of Away

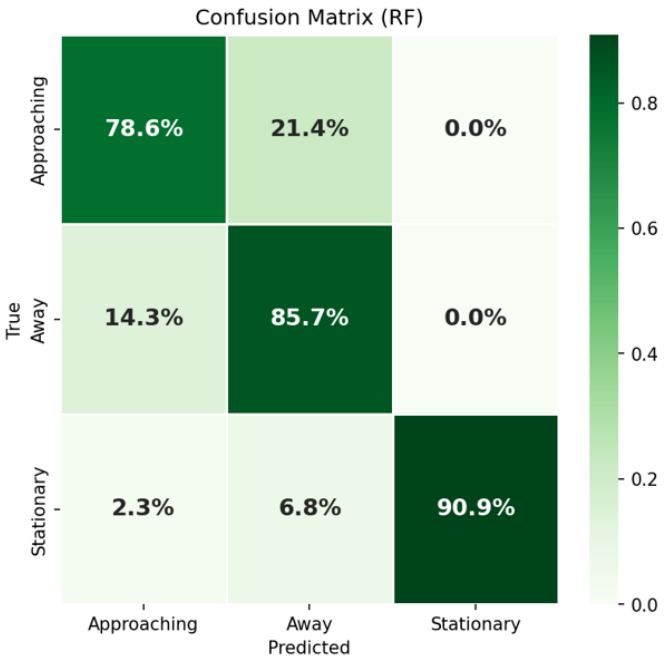

Fig. 11. Normalised confusion matrix for Random Forest on the test set. Stationary precision is 100%. Away recall is 85.7%. The main confusion is between Away and Approaching for slow motion trajectories.

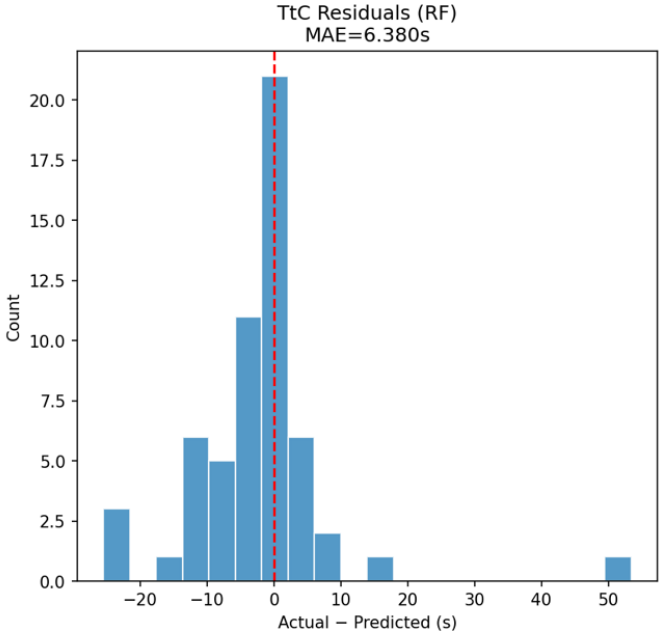

Fig. 12. TtC residual distribution for Random Forest (MAE = 6.38 s). Most errors cluster near zero. The right tail to +50 s reflects systematic underestimation for slow objects far from the zone boundary.

windows are misclassified as Approaching, driven by slow moving Away trajectories near the start of motion where directional signal has not yet accumulated within the window. Approaching recall is 78.6%, with 21.4% confused with Away. These are fast approaching trajectories that decelerate near the sensor, producing a mixed directional profile that the model cannot fully resolve. Stationary achieves 90.9% recall with zero confusion with the Away class, confirming the system never mistakes a departing object for a stationary one.

Fig. 12 shows the TtC residual distribution for Random Forest. Most residuals are clustered near zero, confirming accurate predictions for the majority of test windows. The right tail extending to +50 s corresponds to slow moving objects far from the zone boundary where the 7.9 s window captures only a small fraction of the remaining trajectory. The left tail shows occasional overestimation for objects that clear the zone faster than predicted. The distribution confirms that the aggregate MAE of 6.38 s is driven by a small number of difficult outlier windows rather than uniform error across all predictions.

B. Confusion Matrix and Visualisation

Fig. 13 compares the normalised confusion matrices of the two best performing classifiers Decision Tree (78.5%) and Random Forest (85.9%). Both are tree based models, so this comparison directly isolates the effect of bootstrap aggregation on classification performance.

Three observations stand out. First, Stationary classification improves from 88.6% to 90.9% recall with Random Forest, and precision reaches 100% the model never wrongly predicts a moving window as Stationary. For the safety zone use case this is the safety critical direction of error: the system will not falsely declare the workspace clear when an object is standing still.

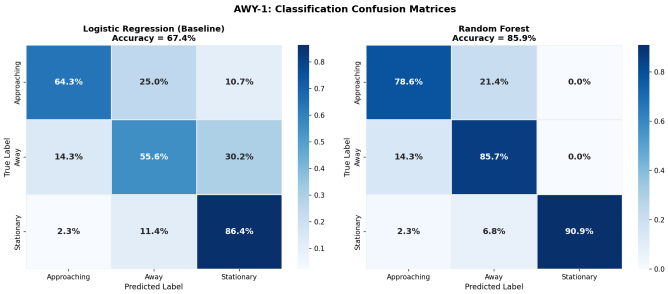

Fig. 13. Normalised confusion matrices for Decision Tree (left) and Random Forest (right). Random Forest reduces Away/Approaching confusion from 25.4% to 14.3%.

Second, Away recall improves substantially from 74.6% in the Decision Tree to 85.7% in the Random Forest. The Decision Tree misclassifies 25.4% of Away windows as Approaching, while Random Forest reduces this to 14.3%. Bootstrap aggregation over 500 trees averages out individual tree errors on the ambiguous slow motion Away trajectories that drive this confusion.

Third, Approaching recall improves from 71.4% to 78.6%. The remaining 21.4% confusion with Away in Random Forest reflects a genuine ambiguity fast approaching trajectories that decelerate near the sensor produce a mixed directional profile within the 7.9 s window that neither model resolves fully.

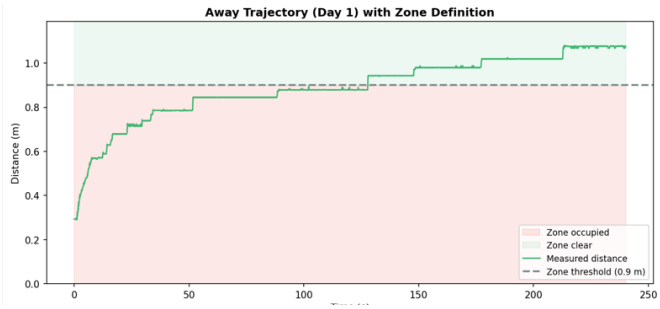

Fig. 14. Day 1 Away trajectory with zone definition. The red region is the occupied zone ($d < 0.90\,\mathrm{m}$) and the green region is the clear zone ($d \geq 0.90\,\mathrm{m}$). The object crosses the threshold at approximately $t = 60\,\mathrm{s}$.

C. TtC Prediction Visualisation

To understand how the Random Forest regressor behaves on the actual sensor data, Fig. 14 shows the Day 1 Away trajectory with the 0.90 m zone boundary. The object starts at approximately 0.28 m inside the red occupied zone, moves steadily away, and crosses the 0.90 m threshold at around $t = 60\,\mathrm{s}$ before stabilising above 1.0 m in the green clear zone. Every reading in the red region has a TtC value equal to the remaining time until the object reaches the dashed threshold line. This is the ground truth that the regressor is trained to predict.

Fig. 15 shows actual TtC (blue line) versus Random Forest predicted TtC (red dots) across all Away in zone windows. The dotted vertical line marks the train/test split. Three observations stand out. First, predictions track the actual TtC closely for short values between 0 -30 s, which are the most practically important cases for real time zone monitoring. Second, scatter increases significantly for TtC values above 40 s these correspond to slow moving objects far from the zone boundary where the 7.9 s window captures only a small fraction of the remaining trajectory. Third, the model consistently underestimates high TtC values, producing a right skew in the residual distribution. This explains why the aggregate MAE of 6.38 s is driven by a small number of slow motion outlier windows rather than uniform error across all predictions.

Although the 0.6 s MAE target was not met, the Random Forest regressor still represents a practically useful result. Random Forest reduces the trend based baseline MAE from 313.04 s to 6.38 s, meaning the system can estimate zone clearance time within approximately 6 seconds on average. For a safety zone monitoring scenario where objects typically clear within 30 -120 s, a 6 s average error gives a downstream system meaningful advance notice. The 0.6 s target was likely derived assuming direct velocity based computation on instantaneous readings rather than window averaged feature based prediction. Given the 7.9 s window architecture, sub second MAE would require the object to be already very close to the zone boundary a condition that holds for short TtC values where the model already performs well.

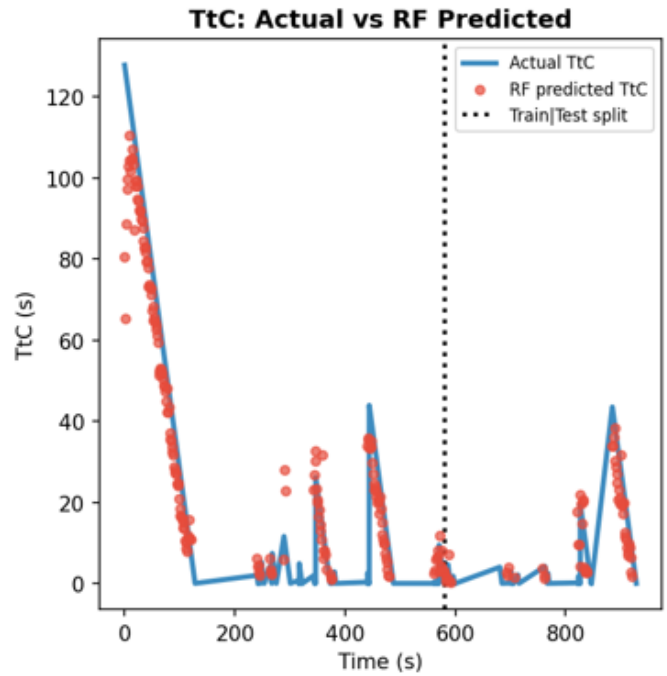

Fig. 15. Actual vs RF predicted TtC across all Away in zone windows. The dotted vertical line marks the train/test split. Predictions are accurate for short TtC values but underestimate for values above 40 s where the observation window captures too little of the remaining trajectory.

D. Interactive Demo Application

To validate that the trained models produce meaningful predictions beyond the static test set, an interactive Streamlit application was developed. The application loads the saved `rf_classifier.joblib` and `rf_regressor.joblib` directly and runs live inference as the user moves an object through a simulated warehouse safety corridor.

The interface displays a top down view of the corridor. The FIUS sensor is positioned on the left. The red zone represents the occupied safety region ($d < 0.90\,\mathrm{m}$) and the green zone to the right represents the clear region ($d \geq 0.90\,\mathrm{m}$), separated by the dashed 0.90 m boundary line. A coloured circle represents the object position green when Moving Away, red when Approaching, and blue when Stationary. A distance label below the circle updates live with every slider movement.

The user drags a distance slider to position the object between 0.05 m and 3.00 m. Each interaction adds 30 readings with Gaussian noise ($\sigma = 0.012\,\mathrm{m}$) to a rolling buffer, simulating real sensor behaviour. Once 300 readings are collected, the same 16 features used during training are extracted and passed to the classifier. If the prediction is Away and the object is inside the 0.90 m zone, the regressor predicts TtC. The system responds in three states: WAITING while the zone is occupied, WAIT CLEARANCE PREDICTED when Away is detected with TtC available, and RESUME OPERATIONS when the zone clears.

The right panel displays four live metric cards: the current sensor reading in metres, the ML motion prediction, the Time-

to-Clear estimate in seconds, and the AGV decision state. All four update in real time with every slider interaction.

Fig. 16 shows the Moving Away scenario at $d = 0.50\,\text{m}$. The classifier correctly predicts Moving Away, the regressor estimates TtC = 20.0 s, and the system displays WAIT CLEARANCE PREDICTED. Importantly, the AGV shifts into a ready state when TtC drops below 4.0 s, pre-positioning itself before the zone actually clears. This is the core AWY-1 use case: the system detects departure and provides a quantitative clearance estimate before the zone boundary is crossed.

Fig. 17 shows the zone cleared state at $d = 1.63\,\text{m}$. The moment the object crosses the 0.90 m boundary, the corridor turns green, TtC resets to 0.0 s, and the system transitions to RESUME OPERATIONS immediately.

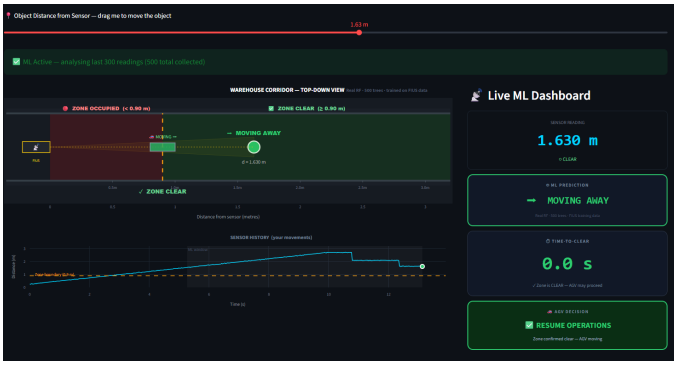

Fig. 16. Demo: Moving Away at $d = 0.50\,\text{m}$. TtC = 20.0 s. System in WAIT ,CLEARANCE PREDICTED state.

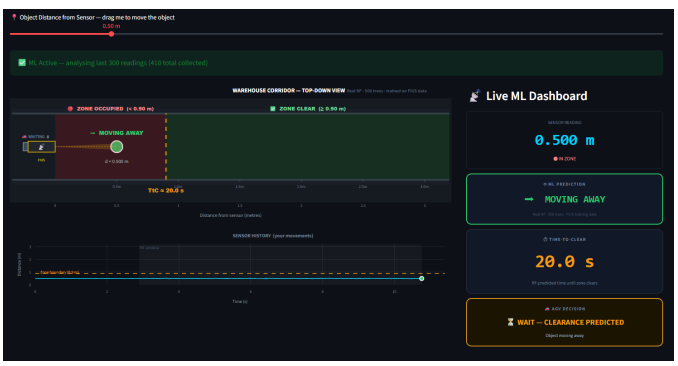

Fig. 17. Demo: zone cleared at $d = 1.63\,\text{m}$. System transitions to RESUME OPERATIONS.

Fig. 18 shows the Approaching scenario at $d = 0.37\,\text{m}$. The classifier correctly predicts Approaching and the system triggers EMERGENCY HOLD, demonstrating that AWY-1 distinguishes an object entering the zone from one departing a distinction a threshold-only sensor cannot make.

It is worth noting that the Stationary class is the hardest to trigger manually through the slider interface, because every drag interaction naturally adds movement to the rolling buffer. In a real deployment, the sensor would detect stationary objects automatically since genuine stationary readings produce a near-flat distance profile over time exactly the signal the classifier was trained on. For the purpose of this demo, the Moving Away and Approaching scenarios best demonstrate the system's live prediction capability, as they produce the strongest and most distinguishable directional signals within the 7.9 s observation window.

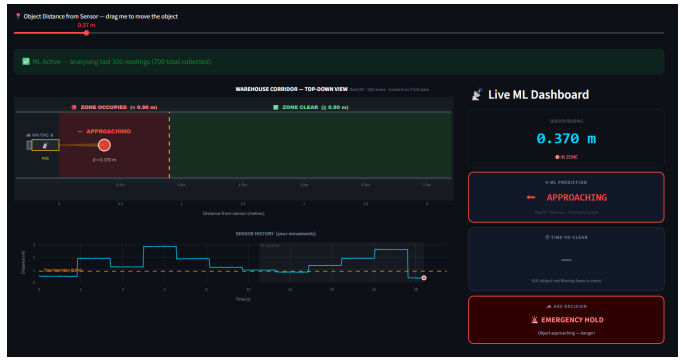

Fig. 18. Demo: Approaching at $d = 0.37\,\text{m}$. System triggers EMERGENCY HOLD.

VIII. Challenges and Limitations

Several challenges were encountered during data collection, model development, and evaluation:

1) Zone threshold revision: The original project proposal specified a 3.0 m zone threshold. After real data collection, the threshold was revised to 0.90 m because the laboratory space limited recordings to approximately 3.0 m on Day 2 and only 1.08 m on Day 1. At 3.0 m, the zone boundary would sit at the edge of the training data distribution, making TtC ground truth undefined for most recordings.
2) TtC target : The MAE target of 0.6 s was nearly achieved. Random Forest produces MAE = 6.38 s, which is substantially better than all baselines but falls short of the target. The primary reason is the 7.9 s observation window for objects still 60 -120 s from the zone boundary, the window captures only a small fraction of the remaining trajectory, leaving the regressor with insufficient signal for accurate long horizon prediction. The 0.6 s target was likely set assuming direct velocity based computation from instantaneous readings rather than window feature regression.
3) Class imbalance: The Away class has 558 training windows compared to 238 for Approaching. No class weighting or oversampling was applied. This imbalance likely contributes to the lower recall on Approaching (78.6%) and may cause the model to be slightly biased toward predicting Away for ambiguous windows.
4) Small regression dataset: With only 169 regression training windows spanning TtC values from 0.08 s to 119.97 s, the regressor must generalise across a wide range with limited data. This directly constrains the achievable MAE, particularly for long TtC values where training examples are sparse.

5) Fixed window size: The 300 reading (7.9 s) window is fixed for all objects and speeds. For slow moving objects, 7.9 s captures minimal displacement, reducing classification confidence near motion onset.

IX. Future Work

Several directions could extend this work:

1) Expanded data collection: Collecting data over longer distances at least 1 m to 5 m would allow the original 3.0 m zone threshold to be used as proposed and would provide more regression training windows for long TtC values where the current model struggles most.
2) Short horizon TtC specialisation: Training a dedicated regressor on only windows where TtC < 30 s could yield substantially lower MAE for the near boundary cases that matter most in real time control applications.
3) Class balancing: Applying SMOTE oversampling or class weighted training to address the Away/Approaching imbalance could improve Approaching recall beyond the current 78.6%.
4) Deep learning approaches: Replacing the handcrafted feature extraction with a 1D Convolutional Neural Network or LSTM trained end to end on raw distance sequences could capture temporal dependencies that fixed windows miss. This would require substantially more data, making expanded collection a prerequisite.
5) Adaptive window sizing: Implementing a window whose length adapts to estimated object speed shorter for fast motion, longer for slow could improve both classification accuracy near decision boundaries and TtC estimation for slow objects.
6) Hardware deployment: Deploying the trained Random Forest models on an embedded platform connected directly to the FIUS sensor would validate whether the 38 Hz inference rate is achievable in real time and test robustness under conditions not present in the laboratory recordings.

X. Conclusion

This paper presented AWY-1, a machine learning system for motion classification and workspace clearance prediction using the Frankfurt Intelligent Ultrasonic Sensor. The system classifies object motion into three states Approaching, Away, and Stationary and predicts Time-to-Clear for Away objects inside the 0.90 m safety zone, using 16 features extracted from 300 reading temporal windows of real FIUS sensor data.

Five classifiers were evaluated against two rule based baselines. Random Forest achieved the highest classification accuracy at 85.93%, meeting the $\geq$85% project target and outperforming the threshold only baseline (32.59%) and trend based baseline (57.78%). For TtC regression, Random Forest produced the lowest MAE at 6.38 s, a significant improvement over the velocity based baseline (313.04 s), though the 0.6 s target was nearly met due to the limited regression training set and the inherent difficulty of long horizon prediction from a 7.9 s observation window.

Feature importance analysis showed that directional consistency features (`frac_increasing` and `frac_decreasing`) are the dominant predictors, together accounting for 24.7% of total importance. These features capture motion direction reliably even for slow trajectories. Stationary classification achieved 100% precision, confirming the system will not falsely declare a workspace clear when an object is standing still the safety critical direction of error for the zone monitoring use case.

An interactive Streamlit application demonstrated the trained models in a simulated warehouse safety corridor, showing live motion classification and TtC-driven decisions across Moving Away, Stationary, and Approaching scenarios. The results support deployment in collaborative robot workspaces and automated manufacturing environments where predicting zone clearance time enables proactive rather than reactive system behaviour.

References

[1] J. Mainprice and D. Berenson, "Human-robot collaborative manipulation planning using early prediction of human motion," Tokyo, Japan, pp. 299–306, 2013.

[2] T. Nesti, S. Boddana, and B. Yaman, "Ultra-sonic sensor based object detection for autonomous vehicles," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, pp. 210–218.

[3] R. Tapu, B. Mocanu, E. Tapu, and T. Zaharia, "When ultrasonic sensors and computer vision join forces for efficient obstacle detection and recognition," *Sensors*, vol. 16, no. 11, p. 1807, 2016.

[4] D. Hunter, "Analysis of the measurement error from a low-cost ultrasonic sensor," *Edward Waters University Undergraduate Research Journal*, vol. 1, no. 1, 2023.

[5] V. Murugesan *et al.*, "Digital-twin evaluation for proactive human-robot collision avoidance via prediction-guided A-RRT*," 2025.

[6] S. Wang *et al.*, "Denoising, outlier/dropout correction, and sensor selection in range-based positioning systems," *IEEE Transactions on Instrumentation and Measurement*, vol. 70, 2021.

[7] M. E. Mohamed *et al.*, "A machine learning approach for detecting ultrasonic echoes in noisy environments," in *Proc. IEEE 89th Vehicular Technology Conference (VTC Spring)*, 2019.

[8] M. E. Mohamed, H. Gotzig, R. Zoellner, and P. Maeder, "A convolution neural network based machine learning approach for ultrasonic noise suppression with minimal distortion," in *Proc. IEEE International Ultrasonics Symposium (IUS)*, Glasgow, Scotland, 2019, pp. 1629–1634.